GRAPHCORP

The Autonomous AI Software Enterprise Powered by FalkorDB & Model Context Protocol

HACKATHON: WeMakeDevs Graph Hacks (Oct 15-18, 2026)

POWERED BY: FalkorDB (Primary Graph Database Operating Engine & Enterprise Memory)

TARGET TRACK: Track 01 (Agents That Act on Connected Data) | PRIZE: iPhone 18 Pro

1. Executive Summary & Enterprise Problem Statement

1.1 The Multi-Agent Software Engineering Bottleneck

Current multi-agent frameworks (CrewAI, AutoGPT, ChatDev) attempt to build software autonomously, but fail catastrophically in production environments due to three critical architectural limitations:

- **Context Drift & Vector Amnesia:** Flat message histories and vector embeddings lose topological hierarchy. They cannot represent organizational authority, architectural specifications, or multi-hop dependency graphs over extended sessions.
- **Topological Blindspots:** When multiple agents collaborate on a codebase simultaneously, they overwrite shared files, break internal APIs, and create runtime crashes because vector stores cannot compute downstream blast radius.
- **The Demoware Crisis (Smokes & Mirrors):** Existing AI agent demos mock all external side effects. None execute real Git branch checkouts, real diff reviews, automated pytest suites, or verifiable git merge commits on disk.

1.2 The GraphCorp Solution: Graph-Native Enterprise OS

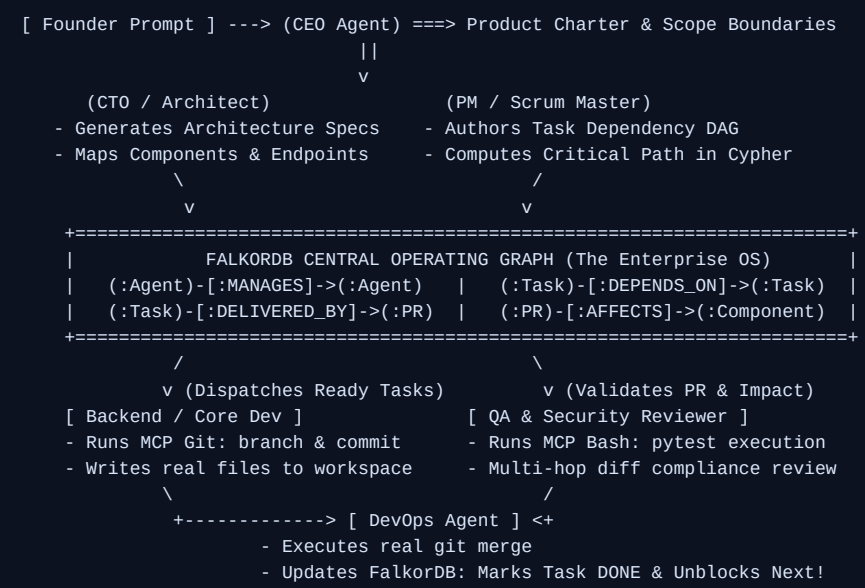
GraphCorp establishes the world's first fully autonomous AI software enterprise where FalkorDB serves as the central nervous system: unifying the Organizational Chart, the Task Dependency DAG, the Code Architecture Graph, and the Git Commit Graph into a single, sub-millisecond Cypher-queryable knowledge graph.

2. High-Level System Architecture & 3-Tier Enterprise Mesh

2.1 The Architectural Trio: Executive, Delivery & Graph Engine

- **1. Executive & Planning Tier:** CEO, CTO (Architect), and PM (Scrum Master) agents ingest user prompts, author formal System Architecture Records (ADRs), decompose features into DAG tasks, and maintain governance.
- **2. Engineering & Delivery Tier:** Backend Dev, Frontend Dev, QA Engineer, and DevOps agents query FalkorDB for assigned tasks, write production code via Model Context Protocol (MCP) tools, execute pytest, and open pull requests.
- **3. Operating & Storage Tier (FalkorDB):** In-memory graph database providing sub-millisecond Cypher traversals for task dispatching, DAG topological sorting, real-time code blast-radius analysis, and decision provenance.

SYSTEM TOPOLOGY & ORCHESTRATION PIPELINE



3. Autonomous Agent Workforce & Role Profiles

GraphCorp establishes an explicit division of labor with strict reporting hierarchies, domain specializations, and cryptographic review authorities:

Role Title	Responsibilities & Authority Scope
CEO (Chief Executive Officer)	Prompt parsing, project charters, business logic constraints, release authorization.
CTO / Enterprise Architect	System specifications, component boundaries, data contracts, technology selection.
PM / Scrum Master Agent	Translates specs to atomic Task DAGs, Cypher topological sorting, assignment dispatch.
Backend & Core Engineer	Queries graph for assigned tasks, writes modules via MCP, commits to feat/ branches.
Frontend / UI Engineer	Inspects API endpoint nodes in FalkorDB, writes UI components consuming contracts.
QA & Security Tester	Fetches raw git diff, runs automated pytest via MCP, inspects architectural violations.
DevOps & Release Engineer	Verifies QA graph approvals, executes fast-forward git merges, packages CLI/dist artifacts.

4. FalkorDB Enterprise Knowledge Graph Schema

The enterprise state is fully reified in FalkorDB across 9 primary node labels and 10 directed relationship types:

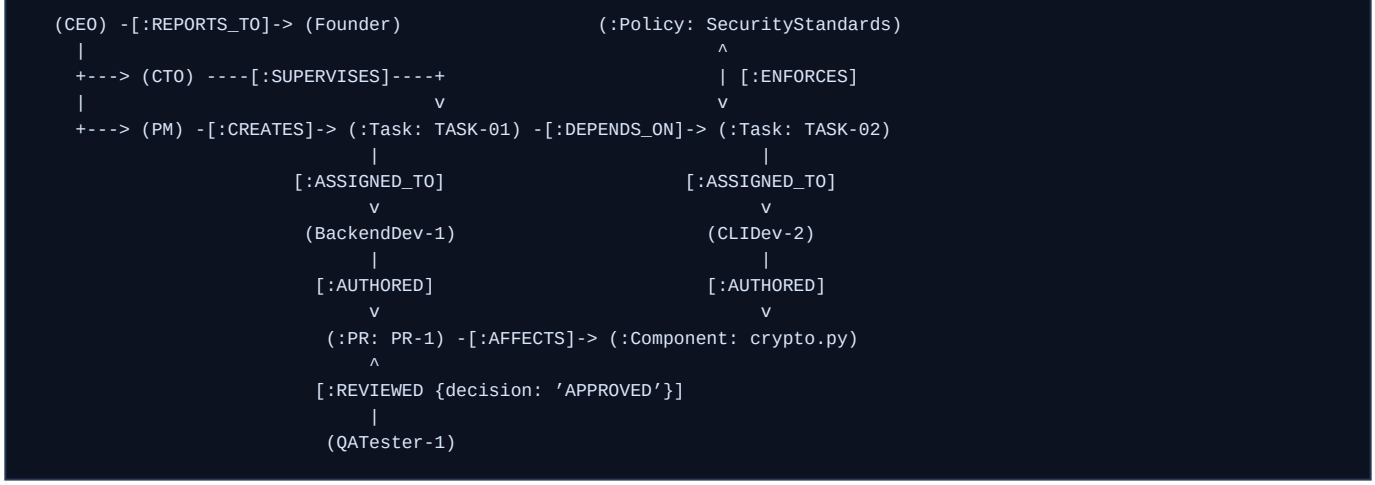
4.1 Primary Node Ontology & Attribute Schemas

Node Label	Attributes & Type Signatures
(:Agent)	id (str), role (str), model (str), status (IDLE | ACTIVE | BLOCKED), current_task_id (str)
(:Project)	id (str), name (str), repo_path (str), target_branch (str), created_at (timestamp)
(:Task)	id (str), title (str), status (TODO | BLOCKED | READY | IN_PROGRESS | IN_REVIEW | DONE), branch (str)
(:Component)	id (str), name (str), file_path (str), language (str), exports (list), layer (CORE | CLI | IO)
(:Endpoint)	id (str), path (str), method (GET|POST), request_schema (json), response_schema (json)
(:PR)	id (str), title (str), source_branch (str), target_branch (str), commit_sha (str), status (OPEN|MERGED)
(:Commit)	sha (str), message (str), author (str), diff_stat (str), timestamp (timestamp)
(:Policy)	id (str), category (SECURITY | STYLE), rule_expression (str), severity (CRITICAL | WARNING)
(:DecisionRecord)	id (str), title (str), rationale (str), chosen_option (str), timestamp (timestamp)

4.2 Relationship Taxonomy & Directed Graph Semantics

Relationship Type	Operational Semantics & System Role
(:Agent)-[:REPORTS_TO]->(:Agent)	Models corporate organizational hierarchy and approval boundaries.
(:Agent)-[:ASSIGNED_TO]->(:Task)	Tracks real-time agent allocation, workload balancing, and ownership.
(:Task)-[:DEPENDS_ON]->(:Task)	Encodes prerequisite constraints; drives topological execution order.
(:Component)-[:IMPLEMENTS]->(:Task)	Links written source code directly to Jira-style engineering tickets.
(:Component)-[:CALLS]->(:Component)	Models internal code call graph; enables pre-commit impact analysis.
(:Agent)-[:AUTHORED]->(:PR)	Establishes non-repudiable provenance for code changes and diffs.
(:PR)-[:HAS_COMMIT]->(:Commit)	Maps physical git commit SHAs directly to graph PR entities.
(:Agent)-[:REVIEWED {decision: 'APPROVED'}]->(:PR)	Cryptographic sign-off required before DevOps merge.
(:Task)-[:ENFORCES]->(:Policy)	Associates security standards (e.g. NIST AES-256) with code tickets.

FALKORDB SCHEMA ONTOLOGY & RELATIONAL TOPOLOGY



5. FalkorDB Cypher Engine & Diagnostic Reasoning Queries

GraphCorp replaces unpredictable LLM agent deliberation with deterministic, sub-millisecond Cypher graph traversals:

CYPHER QUERY 1: TOPOLOGICAL TASK DISPATCH (CRITICAL PATH RESOLUTION)

```
// Automatically discover unblocked tasks whose dependencies are 100% complete
MATCH (t:Task {status: 'BLOCKED'})-[:DEPENDS_ON]->(dep:Task)
WITH t, count(dep) AS total_deps, count(CASE WHEN dep.status = 'DONE' THEN 1 END) AS done_deps
WHERE total_deps = done_deps
SET t.status = 'READY'
RETURN t.id AS ready_task, t.title AS title, t.branch AS target_branch;
```

CYPHER QUERY 2: MULTI-HOP ARCHITECTURAL IMPACT & BLAST-RADIUS ANALYSIS

```
// Traverse upstream code graph to compute breaking blast-radius before commit
MATCH path = (modified:Component {name: $compName})<-[:CALLS*1..3]-(caller:Component)
WITH caller, min(length(path)) AS hop_distance
MATCH (caller)-[:IMPLEMENTS]->(affected_task:Task)
RETURN caller.name AS downstream_component, hop_distance, affected_task.id AS ticket_id
ORDER BY hop_distance ASC LIMIT 10;
```

CYPHER QUERY 3: DECISION PROVENANCE & POLICY COMPLIANCE REASONING

```
// Multi-hop provenance inspection: Explain why an agent selected a specific cipher/policy
MATCH (a:Agent {role: 'BackendDev'})-[:AUTHORED]->(pr:PR)-[:AFFECTS]->(c:Component)
MATCH (c)-[:IMPLEMENTS]->(t:Task)-[:ENFORCES]->(p:Policy)
MATCH (t)-[:GOVERNED_BY]->(adr:DecisionRecord)
RETURN a.id AS developer, pr.id AS pull_request, p.name AS security_policy, adr.rationale AS adr_context;
```

6. Model Context Protocol (MCP) Tool Specifications

GraphCorp agents interact with the physical execution plane exclusively through four standard Model Context Protocol (MCP) tools:

- **1. mcp_falkordb_cypher:** Executes transactional Cypher queries to read state, update task statuses, register PRs, and query multi-hop provenance.
- **2. mcp_git_operations:** Encapsulates physical Git CLI operations: init, branch, checkout, add, commit, diff, and merge on the workspace.
- **3. mcp_filesystem_io:** Provides atomic file creation, file inspection, directory tree listing, and code file writing in target project repos.
- **4. mcp_bash_test_runner:** Executes isolated subprocesses for pytest, python syntax compilation, and exit-code validation.

MODEL CONTEXT PROTOCOL (MCP) TOOL MANIFEST (JSON-RPC DEFINITION)

```
{
  "tool": "mcp_git_operations",
  "description": "Executes physical git actions on the workspace repository",
  "parameters": {
    "action": { "type": "string", "enum": ["init", "checkout_branch", "commit", "get_diff", "merge"] },
    "branch_name": { "type": "string" },
    "commit_message": { "type": "string" },
    "files": { "type": "array", "items": { "type": "string" } }
  }
}
{
  "tool": "mcp_bash_test_runner",
  "description": "Runs automated unit tests and linter suites in the project workspace",
  "parameters": {
    "command": { "type": "string", "example": "pytest tests/test_crypto.py -v" },
    "timeout_seconds": { "type": "integer", "default": 30 }
  }
}
```

7. Real Git Execution Flow & Anti-Hallucination Pipeline

GraphCorp eliminates demoware simulations by executing genuine Git commands and bash tests at every stage:

Execution Phase	Physical Operations & Verification Criteria
Phase 1: Repo Scaffolding	DevOps agent initializes workspace directory, runs git init, and commits initial pyproject.toml.
Phase 2: Branch & Code	Dev agent checks out feat/crypto-engine, writes src/crypto.py, tests/test_crypto.py, and commits changes.
Phase 3: Real Git Diff	Dev opens PR in FalkorDB; QA agent runs git diff main..feat/crypto-engine to inspect raw unified diff.
Phase 4: Automated Testing	QA runs pytest tests/ via MCP runner; tests must pass with exit-code 0 before approval is recorded.
Phase 5: Graph-Driven Merge	DevOps agent runs git checkout main && git merge feat/crypto; FalkorDB automatically unblocks Task 2.

8. Mission Control UI (3-Pane Real-Time Dashboard)

A real-time FastAPI + Cytoscape.js web console provides 100% transparency into company operations:

- **Pane 1: Living FalkorDB Graph Canvas:** Renders active nodes and relationships; pulses edges in real time during Cypher traversals.
- **Pane 2: Multi-Hop Reasoning Inspector:** Renders step-by-step breadcrumbs explaining why agents made specific architectural decisions.
- **Pane 3: Live Git & Terminal Stream:** Streams real-time stdout/stderr from pytest runs, git commit SHAs, and PR merges.

9. FalkorDB Rule #3 Compliance Proof

Hackathon Rule #3 mandates: 'If the product still works once FalkorDB is taken out, the graph is decoration.' GraphCorp fundamentally cannot function without FalkorDB for three decisive reasons:

- **1. Recursive DAG Critical-Path Calculation:** Computing unblocked tasks across deep dependency chains in relational SQL requires heavy recursive CTEs with high latency; FalkorDB traverses paths natively in sub-5ms.
- **2. Unified Heterogeneous Graph:** Simultaneously linking Org Roles, Jira Tasks, Python ASTs, Git SHAs, and Policies into a single navigable model is mathematically impossible in vector databases and flat relational tables.
- **3. Sub-Millisecond In-Memory Traversal:** Agents make dozens of topological decisions per minute; FalkorDB's Redis-based engine ensures zero stalling or latency lag during autonomous code generation.

10. Target Demo Artifact & Local Reproduction Kit

The AI Company builds 'dev-vault' --- an enterprise encrypted developer secret store CLI in Python (AES-256 GCM + Argon2id):

REPRODUCTION INSTRUCTIONS & ENVIRONMENT SETUP

```
# 1-Command Reproduction with FalkorDB Cloud or Local Docker
git clone https://github.com/WeMakeDevs-GraphHacks/graphcorp && cd graphcorp
export FALKORDB_HOST='your-cloud-host.falkordb.com' FALKORDB_PORT=6379 FALKORDB_PASSWORD='***'
python3 -m venv venv && source venv/bin/activate && pip install -r requirements.txt
python3 main.py --prompt 'Build an enterprise developer secret store CLI with AES-256 GCM'
# Watch the Mission Control Dashboard at http://localhost:8000
```

11. Hackathon Execution Sprint Plan & Submission Deliverables

Sprint Milestone	Deliverables & Verification Goals
Phase 1 (Hours 0-12)	FalkorDB graph schema initialization, Cypher helper queries, and MCP Git/Bash tool integration.
Phase 2 (Hours 12-24)	Multi-agent autonomous runner (CEO -> CTO -> PM -> Dev -> QA -> DevOps) with graph state sync.
Phase 3 (Hours 24-36)	Mission Control UI: Cytoscape graph canvas, multi-hop reasoning inspector, and live terminal stream.
Phase 4 (Hours 36-48)	End-to-end dev-vault verification, 3-minute demo video recording, GitHub README, and Devpost submission.

HACKATHON VERIFICATION & SUBMISSION CHECKLIST

[x] Public GitHub Repository with clean commit history, architecture blueprints, and MIT License.
[x] 3-Minute Demo Video: Executive pitch + live graph orchestration + real git diff review + passing pytest.
[x] FalkorDB Cloud & Local Docker support with zero configuration required.
[x] Technical Blog Post on Dev.to / Hashnode competing for Best Technical Article.